

PPAU VLSI - Laboratorium 2

Podczas tych zajęć poddane będą analizie rozrzuty technologiczne, jakie występują w zintegrowanych układach elektronicznych. W pierwszych krokach będą analizowane rozrzuty prądu klasycznego źródła prądowego i parametry od jakich one zależą, a następnie zbudowany będzie 5-cio bitowy przetwornik cyfrowo-analogowy DAC z ważonymi prądami.

Wykorzystywane środowisko to IIC-OSIC-TOOLS ([github](#)), a używany PDK to sky130A od SkyWater Foundries ([pdk-documentation](#)). Sama biblioteka sky130 ma wiele przykładowych komórek bazowych jak i testów dla różnych układów. Dostęp do wszystkich można znaleźć w różnych folderach tej biblioteki, gdzie przykładowo `sky130_tests/` zawiera wiele przykładowych bloków symulacji dla różnych układów, w tym wykorzystywane na tym laboratorium analizy parametryczne jak Monte Carlo. Ponieważ symulacje nie są kontrolowane za pomocą interfejsu graficznego, należy zapoznać się ze składnią poleceń ngspice (do sprawdzania składni i przykładów - [manual](#))

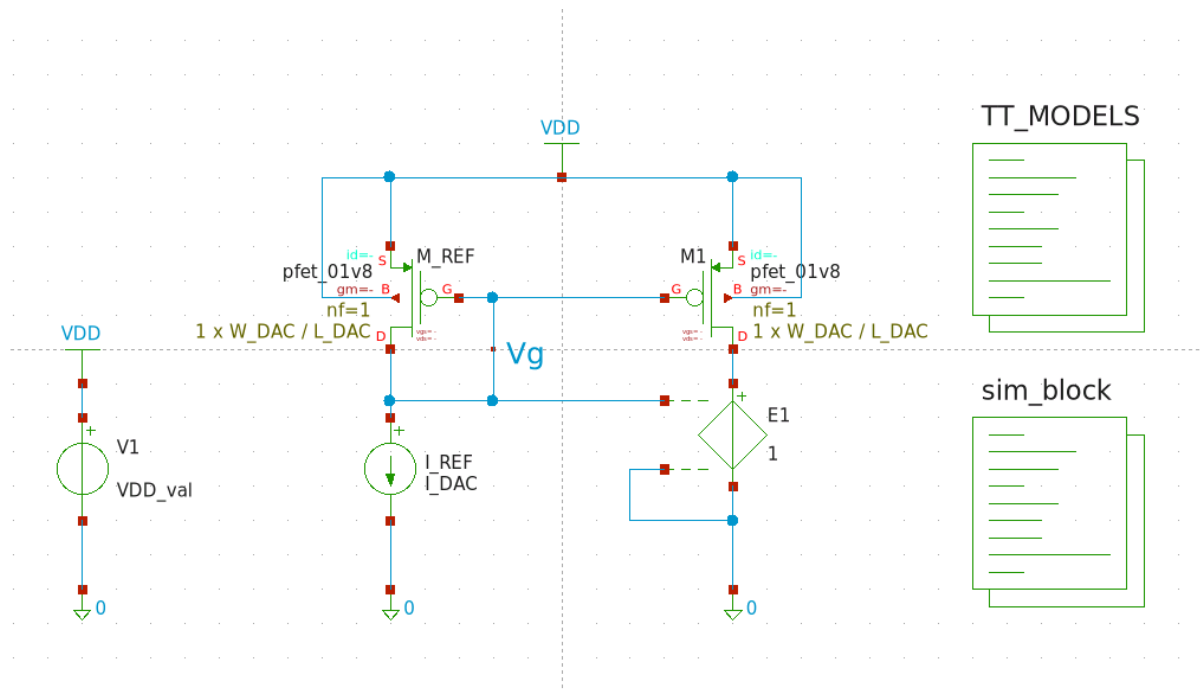
1. Rozrzuty w zintegrowanych układach logicznych

1.1. Przygotuj schemat układu źródła prądowego z Rys. 1.1. i nazwij schemat *CURRENT_SOURCE*. Jako tranzystorów użyj **pfet_01v8** z biblioteki **sky130_fd_pr** (fd - The SkyWater Foundry, pr - Primitive Cells; więcej o nazewnictwie w sky130 PDK można znaleźć [tutaj](#)), a dla całego schematu użyj parametrów:

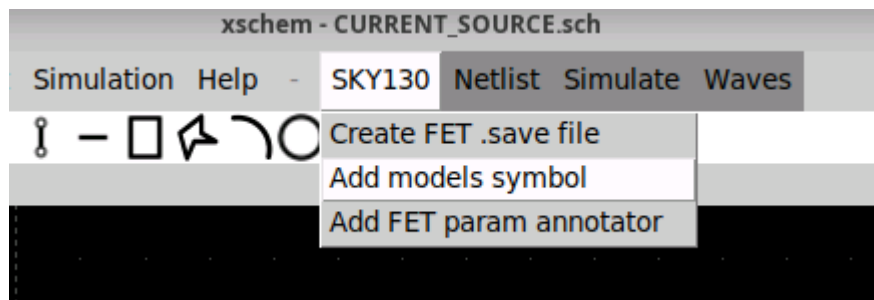
- szerokość i długość kanału mosfetów -> odpowiednio **W_DAC** i **L_DAC**
- Źródłu prądowemu -> parametr **I_DAC**
- Źródłu napięcia -> parametr **VDD_val**

Aby w realizowanych analizach wyeliminować błąd systematyczny związany z różnymi napięciami V_{ds} tranzystorów, użyj źródła napięciowego sterowanego napięciem - **vcvs** - z domyślnej biblioteki **devices** z ustawionym wzmocnieniem 1. Elementy widoczne na schemacie takie jak źródła prądowe, napięciowe, symbole zasilania, uziemienia czy bloki kodu także można znaleźć w tej bibliotece. Napięcie zasilania VDD powinno być równe 1.8V.

Wykorzystany blok kodu o nazwie **TT_MODELS** to gotowy blok, który jest łatwo i szybko dostępny wybierając na pasku menu `SKY130 -> Add model symbol` (Rys 1.2.) i dla parametrów typowych nie musi być edytowany.



Rys 1.1. Schemat źródła prądowego.



Rys 1.2. Dodanie bloku kodu zawierającego bibliotekę.

1.2. Następnie sprawdzimy punkt pracy tranzystorów PMOS.

Dla trzech różnych wymiarów tranzystorów W / L odpowiednio:

- 0.42 / 0.15 [$\mu\text{m} / \mu\text{m}$]
 - 5 / 3.5 [$\mu\text{m} / \mu\text{m}$]
 - 3.5 / 5 [$\mu\text{m} / \mu\text{m}$]
- oraz dwóch różnych prądów I_{DAC} :
- 100 [nA]
 - 10 [μA]

Wyznacz napięcia V_{gs} i V_{th} tranzystora M1. Uzyskane wyniki zanotuj w sprawozdaniu.

1.3. Aby przeprowadzić analizy Monte Carlo musisz wskazać środowisku symulacyjnemu modele elementów technologicznych, w których będą występowały parametry związane z rozrzutami technologicznymi. Modele te są identyczne jak te stosowane do tej pory, a różnią się jedynie dodatkowymi zmiennymi odpowiedzialnymi za rozrzuty.

W tym celu wystarczy zmienić parametr, z którym wywoływana jest biblioteka w bloku TT_MODELS. W lini, w której załączana jest biblioteka (`.lib $::SKYWATER_MODELS/sky130.lib.spice tt`) zamień `tt` na wybrany corner. Na razie zastąp to przez `tt_mm`. Warto też zobaczyć co dokładnie znajduje się w załączanym pliku.

```
cd /foss/pdks/sky130A/libs.tech/combined
view sky130.lib.spice
```

1.4. W środku pliku `sky130.lib.spice` można zauważyć dużo zdefiniowanych cornerów wraz z informacją, które biblioteki z tego folderu są dołączane. Dla każdego corneru ustawiane są dwa parametry - przejrzyj te cornery, szczególnie **tt**, **tt_mm**, **mc**, i zastanów się do czego są te parametry i co oznaczają.

1.5. W kolejnym kroku skonfiguruj symulację Monte Carlo. W Xschem jest to realizowane przez załączanie odpowiednich plików bibliotecznych (blok TT_MODELS) oraz pętli kodu ngspice. Przykład takiej pętli jest dostępny na schemacie `sky130_tests/montecarlo_mismatch_sim.sch` w bloku kodu "NGSPICE". Alternatywą dla bloku TT_MODELS jest dostępny symbol `sky130_fd_pr/corner.sym`, w którym zmieniając wyłącznie parametr *corner* można dołączyć odpowiednie pliki. Blok TT_MODELS ze względu na to że zawiera pole kodu zamiast pojedynczego parametru, przez co możemy je wykorzystać do definicji idealnego modelu - który nie będzie podlegał rozrzutom.

Dane można zbierać do plików za pomocą `echo [text and vectors] >> [file_path/filename]` lub przykładowo: polecenia `wrdata`, `write` (więcej informacji o formacie zapisu oraz o sposobach zapisu danych do pliku można znaleźć w [ngspice manual](#)). **TU MOŻE COŚ ZMIEŃIĆ** Aby wyświetlić wyniki w formie graficznej zebrane dane w plikach można wyeksportować i wyplotować za pomocą wbudowanego `gnuplot` bezpośrednio z otwartej konsoli dockera lub ngspice'a. Dzięki dostępowi do konsoli dockera można pisać skrypty .sh, .tcl, które z zapisanych danych będą tworzyć wykresy i zapisywać je do plików graficznych.

1.6. Rozrzuty w PDK Sky130 są sterowane globalnie poprzez parametry **mc_mm_switch** i **mc_pr_switch**. Jeśli chcesz, aby dany tranzystor (np. lustro referencyjne M_REF) nie podlegał zmianom związanym z mismatchem czy procesem należy stworzyć idealny model naszego elementu, który nie będzie brał pod uwagę globalnie zdefiniowanych rozrzutów. Sprowadzać się to będzie do stworzenia podobwodu, który będzie miał wyłączone parametry kontrolujące mismatch i process (wartości tych parametrów dla podobwodów są traktowane lokalnie, więc nie trzeba się martwić o nadpisywanie wcześniej dodanych parametrów). Nazwa modelu powinna dokładnie opisywać element i jego funkcję. Warto pamiętać przyjętej konwencji do nazywania elementów dla danej biblioteki - w naszym przypadku, dla przykładowo PMOS-a, wygląda to następująco: `sky130_fd_pr__` to fragment wskazujący na bibliotekę, a `pfet_01v8` to nazwa modelu. Można zweryfikować jak wyglądają definicje instancji sprawdzając netlistę (skrót klawiszowy *Shift* + *N* lub *N*, następnie `Simulation -> Edit Netlist`).

Dzięki zachowaniu tej składni, po dodaniu fragmentu definiującego podschemat, wystarczy zamienić parametr *model* danego elementu (w naszym przypadku PMOS-a) bez tworzenia nowego symbolu dla tego modelu.

1.7. W naszych następnych symulacjach chcemy analizować rozrzuty pochodzące jedynie od tranzystora M1. Ponieważ domyślnie włączenie `.param mc_mm_switch = 1` aktywuje losowanie parametrów dla wszystkich elementów na schemacie, aby zasymulować idealne źródło referencyjne M_REF (bez rozrzutów) stworzymy dla tego schematu wspomniany subcircuit - pamiętając o konwencji nazywania elementów w bibliotece.

```
* disable pfet_ideal from mismatch/process
.subckt sky130_fd_pr__pfet_ideal D G S B W=0.42 L=0.15
+ nf=1 ad='int((1 + 1)/2) * {W} / 1 * 0.29' as='int((1 + 2)/2) * {W} / 1 * 0.29'
+ pd='2*int((1 + 1)/2) * ({W} / 1 + 0.29)' ps='2*int((1 + 2)/2) * ({W} / 1 +
0.29)'
```

```
+ nrd='0.29 / {W}' nrs='0.29 / {W}' sa=0 sb=0 sd=0 mult=1
.param mc_mm_switch=0
.param mc_pr_switch=0
XIDEAL D G S B sky130_fd_pr__pfet_01v8 W={W} L={L}
+ nf={nf} ad={ad} as={as} pd={pd} ps={ps}
+ nrd={nrd} nrs={nrs} sa={sa} sb={sb} sd={sd} mult={mult}
.ends
```

Ten fragment można dodać do któregoś z bloków kodu, ale zalecane jest dodanie go do bloku TT_MODELS gdzie oprócz dodawania plików cornerowych będziemy także uwzględniać efekty mismatchu i procesu. Zamień teraz model tranzystora M_REF z `pfet_01v8` na `pfet_ideal`. Jeżeli do tej pory wszystko poprawnie zostało zrobione - symulacja będzie uwzględniać rozrzuty wyłącznie pochodzące od tranzystora M1.

1.8. Przeprowadź analizy Monte Carlo (100 prób dla każdego z przypadków) dla wariantów prądów oraz wymiarów tranzystorów M_REF i M1 podanych uprzednio (pamiętaj o uwzględnieniu wyłącznie M1 w rozrzutach).

Uzyskane wyniki σ /AVG (sigma/wartość średnia) zanotuj w sprawozdaniu. Uzupełnij wszystkie brakujące pola w tabeli.

1.9. Odpowiedz na poniższe pytania:

- dla tych samych wymiarów tranzystorów a różnych prądów znacząco różnią się wartości σ /AVG?
- dla tych samych zarówno powierzchni tranzystorów jak i ich prądów wartości σ /AVG różnią się pomiędzy sobą?
- wskaż dwa z analizowanych przypadków, które dowodzą różnych kontrybutorów rozrzutów (kontrybucja od napięcia progowego, kontrybucja od współczynnika prądowego).

1.10. Sprawdź stopień kontrybucji rozrzutów prądu wyjściowego (σ /AVG) dla trzech przypadków (rozważ tylko przypadek $3.5 \mu\text{m} / 5 \mu\text{m}$ oraz $I_{\text{DAC}} = 10 \mu\text{A}$; Tylko pochodzące z mismatchu

`.mc_mm_switch=1 .mc_pr_switch=0`):

- uwzględniamy tylko rozrzuty tranzystora M1 (*model* M_REF ustaw jako `pfet_ideal`),
- uwzględniamy tylko rozrzuty tranzystora M_REF (*model* M1 ustaw jako `pfet_ideal`),
- uwzględniamy rozrzuty obu tranzystorów (*model* obu tranzystorów to `pfet_01v8`).

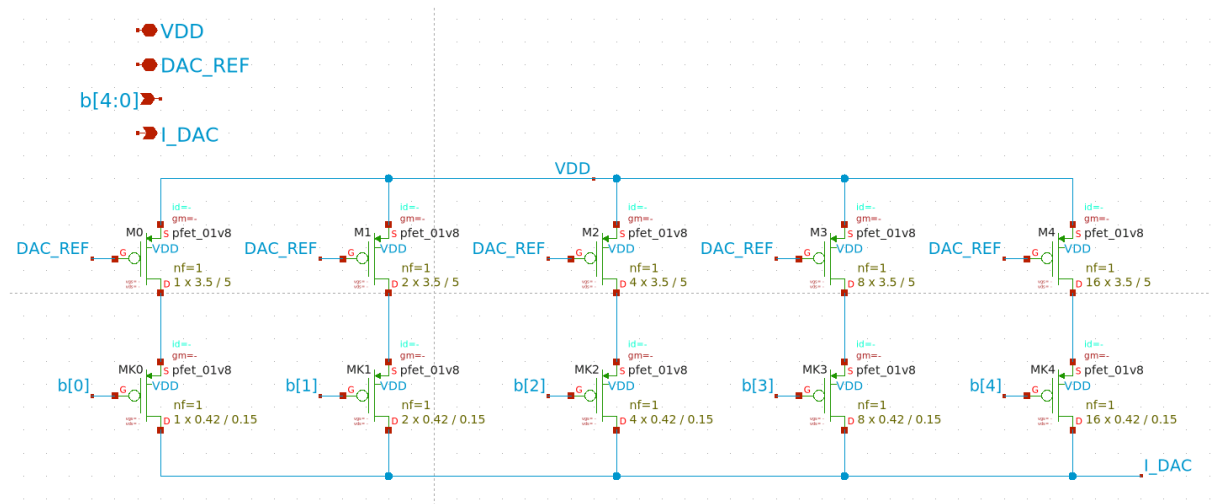
Wyniki zapisz w sprawozdaniu. Czy są one zgodne z Twoimi przypuszczeniami? Dlaczego?

1.11. Powtórz symulacje z poprzedniego punktu uwzględniając proces.

Uwzględnij rozrzuty Process i Mismatch (`.mc_mm_switch=1 .mc_pr_switch=1`). Wyjaśnij występujące różnice.

2. Schemat 5-bitowego DAC-a

2.1. W dalszej części ćwiczenia będziesz budować 5-bitowy przetwornik DAC. Dlatego utwórz schemat i jego symbol zgodnie z Rys. 2.1. Nazwij go *DAC_CORE_5_Bit*. Źródła prądowe M0 - M4 mają wymiary $W/L = 3.5 \mu\text{m} / 5 \mu\text{m}$, zaś wymiar kluczy MK0 - MK4 to $W/L = 0.42 \mu\text{m} / 0.15 \mu\text{m}$.



Rys 2.1. Schemat 5-bitowego DAC-a.

2.2. Odpowiedz na pytanie w sprawozdaniu:

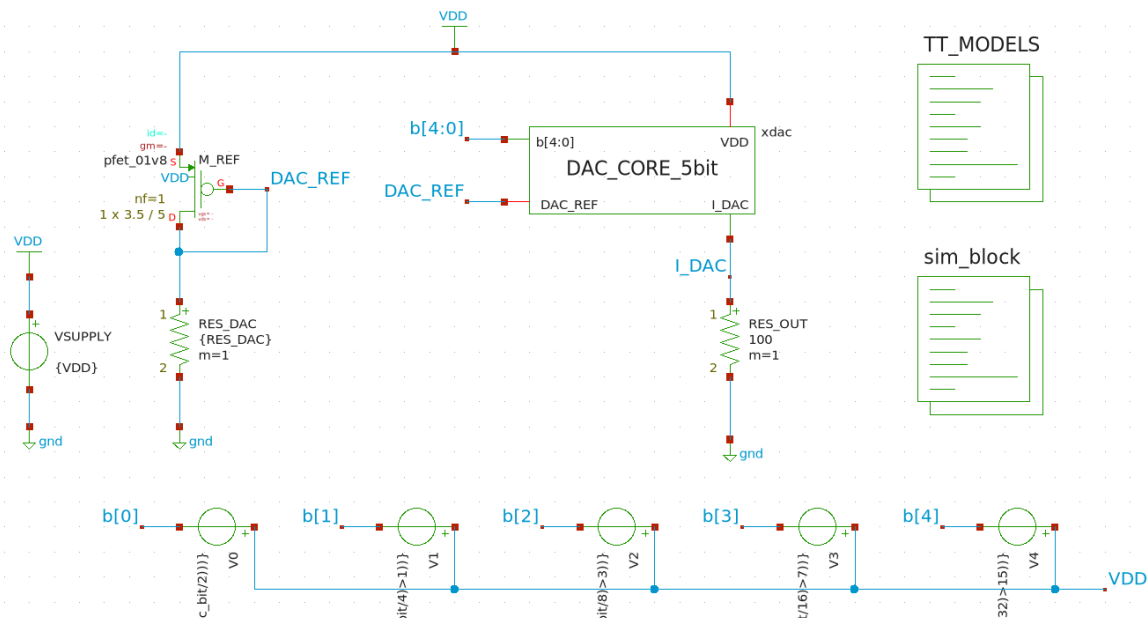
Czy istotnym jest by liczba tranzystorów pracujących jako klucze odpowiadała liczbie tranzystorów pracujących jako źródła prądowe? Uzasadnij dlaczego.

2.3. Zbuduj schemat na potrzeby symulacji tak jak na Rys. 2.2. Ustaw następujące parametry:

- prąd źródła prądowego ustaw za pomocą rezystora RES_DAC - tak, aby wynosił on 10 μ A,
- rezystancję R_OUT na wartość 100 ,
- wymiary tranzystora M_REF, W/L = 3.5 μ m / 5 μ m.

Rezystory oraz źródła napięciowe widoczne na schemacie pochodzą z biblioteki **devices**. Zwróć uwagę na podłączenie źródeł napięcia stałego V0 - V4. Są one podpięte między wejścia b[4:0] a szynę napięcia zasilającego VDD. Wynika to z faktu, że wyjściowe napięcia tych źródeł przyjmują wartości 0V bądź VDD, a tranzystory DAC-a to tranzystory typu PMOS (active low). W kolejnym kroku należy w odpowiedni sposób zdefiniować wartości napięć stałych tak, by móc przeprowadzić symulację charakterystyki prądu wyjściowego DAC-a w funkcji jego wejść b[4:0]. W tym celu można skorzystać ze źródła napięcia stałego `vsource`, w którym w pozycji *value* należy wpisać:

- dla V0: `{ VDD * ((dac_bit - 2*int(dac_bit/2))) }`
- dla V1: `{ VDD * ((dac_bit - 4*int(dac_bit/4)>1)) }`
- dla V2: `{ VDD * ((dac_bit - 8*int(dac_bit/8)>3)) }`
- dla V3: `{ VDD * ((dac_bit - 16*int(dac_bit/16)>7)) }`
- dla V4: `{ VDD * ((dac_bit - 32*int(dac_bit/32)>15)) }`



Rys 2.2. Schemat układu do symulacji.

W powyższym zapisie VDD i dac_bit oznaczają parametry (zmiennne projektowe) odpowiednio napięcia zasilania 1.8 V i cyfrowego słowa na wejściach b[4:0], które zmienia się w zakresie 0-31. Powyższą składnia ma za zadanie generować napięcia na wyjściach źródeł V0 - V4 tak by dla zmian dac_bit w zakresie 0-31 we właściwy sposób ustawić wejścia b[4:0] (od stanu 00000 do stanu 11111 włącznie).

Przetestuj działanie źródła V0 dla wartości parametru dac_bit z zakresu 0-5 i zanotuj napięcie tego źródła dla tego zakresu. Możesz zasymulować działanie układu poprzez wektory ngspice lub wykorzystać stworzony układ (podpowiedź: wykorzystaj odwołanie do wartości elementu @v0[dc])

Możesz wykorzystać Uzyskane wyniki zapisz w sprawozdaniu.

2.4. TBD

Zapisz w sprawozdaniu:

- Rezystancję RES_DAC, dla której źródło prądowe generuje 10 μA
- największy generowany przez przetwornik prąd $I_{\text{DAC MAX}}$.

Za pomocą analizy DC wyznaczyć charakterystykę przejściową DAC-a dla zmian słowa wejściowego 0-31 z krokiem 1. Umieścić ją w sprawozdaniu. Aby zweryfikować poprawność swojej odpowiedzi na pytanie z podpunktu 2.2. - wyznaczyć również charakterystykę przejściową zmodyfikowanego DAC-a, w którym każdy z kluczy MK0 - MK4 zastąpić pojedynczym tranzystorem (co sprowadza się do zmiany wartości parametru mult na 1 dla każdej gałęzi). Umieścić obie charakterystyki w sprawozdaniu. Czy Twoje przypuszczenia były poprawne?

UWAGA: „napraw” schemat przetwornika przed przejściem do kolejnych kroków - liczba kluczy powinna się zgadzać z liczbą źródeł prądowych w danej gałęzi.

2.5. TBD

Wyznacz, ile wynoszą rozrzuty prądów generowanych przez DAC-a (σ) oraz ile wynosi wartość σ/AVG . Pod uwagę weź jedynie Mismatch oraz rozrzuty samego bloku DAC_CORE_5_Bit. Rozpatrz wartości zmiennej dac_bit: 1, 2, 4, 8, 16, 31.

Czy dostrzegasz jakieś związki pomiędzy poszczególnymi wynikami a liczbą załączonych tranzystorów?
Jakie są to relacje?

2.6. TBD

Przeprowadź analizę MC (tylko bloku DAC i jedynie z opcją Mismatch) charakterystyk wyjściowych DAC-a. Ile wynosi wartość średnia, maksymalna i minimalna generowanego prądu dla `dac_bit = 31`?

PONIŻEJ ZAKUTALIZOWAĆ DLA NGSPICE'A

2.7. Teraz skonfigurujesz środowisko by móc przeprowadzić analizy brzegowe. W tym celu w oknie Data View (ADE Assembler) zaznacz opcję Corners, zmień profil symulacji na Single Run, Sweeps and Corners a następnie kliknij w pole Click to add corner (Rys. 2.3). Pojawi się okno Corners Setup (Rys. 2.3). Obecnie okno zawiera tylko jeden profil symulacyjny - jest to profil nominalny.

2.8. Twoim zadaniem będzie zdefiniowanie analiz brzegowych. Standardowo realizuje się to w oparciu o dokumentację dostawcy technologii (na zajęciach przeprowadzisz kilka z tych analiz). W tym celu w oknie Corners Setup dodaj cztery kolumny (przycisk Add New Corner w głównym menu okna Corners Setup). Następnie w polu Model Files kliknij opcję Click to add. W oknie, które się pojawi, wybierz opcję Import From Tests. Zostanie wczytana biblioteka modeli, z której obecnie korzystasz (Rys. 2.4). Następnie kliknij OK.

2.9. W dalszej kolejności wypełnij tabelkę okna Corners Setup jak na Rys. 2.5 (by wypełnić dane pole należy kliknąć na nie dwukrotnie). Nazwy, które tutaj wpisujesz, oznaczają definicję analiz brzegowych, które zaleca dostawca technologii. Odnoszą się one do warunków brzegowych procesu, takich jak najmniejsza/największa ruchliwość nośników, najmniejsze/największe napięcie progowe. Zauważ, że dodatkowo możesz zdefiniować zakres zmian temperatury czy też parametrów w symulowanym obwodzie. W tej części ćwiczenia pozostaw jednak te pola puste. Po zdefiniowaniu analiz brzegowych kliknij OK.

2.10. TBD

Uruchom symulacje i zanotuj typowy, maksymalny i minimalny prąd jaki generuje DAC dla `dac_bit = 31`. Porównaj go z przeprowadzonymi wcześniej analizami MC. Które wyniki są mniej korzystne i dlaczego?

3. Layout 5-bitowego DAC-a

TBD

4. Possible simulation errors

TBD

TODO LIST

1. Znaleźć sposób na uwzględnienie rozrzutów tylko pojedynczego tranzystora i uzupełnić punkty 1.5 - 1.7 (znalazłem sposób - trzeba uzupełnić instrukcję) - zrobione (chyba dobrze)
 - To co Defaultowo jest przez `SKY130->Add models symbol` może być zmienione poprzez modyfikację `/foss/pdks/sky130A/libs.tech/xschem/xschemrc`, ale to nie będzie permanentna zmiana, a wyłącznie na obecną sesję dockera, do stałej zmiany można dodać plik `/foss/designs/xschemrc` **POD WARUNKIEM że będziemy startować każdy schematic z folderu**

/foss/designs co przy dużej ilości podfolderów i plików, czy kilkuosobowej pracy nad jednym projektem, może być niewygodne (trzeba być zapoznanym z drzewem folderów)

- dla bardzo małych prądów, mogą powstawać niedokładności między dużą ilością powtórzeń.

Czemu? Może to przez niedokładności w modelowaniu? to raczej nie jest problem tylko z .subckt - dla pfetów prosto z sky130 też tak to wygląda - sprawdzić jeszcze trzeba i się zastanowić

- Przeprowadź analizę MC z punktu 1.8 i zapisz wyniki (kod do symulacji gotowy - ustawić M_REF jako mosfet idealny) - zrobione
- Zaktualizuj polecenie 1.10 uwzględniając odpowiednie parametry i sposoby na rozrzucanie pojedynczego tranzystora - zrobione
- Wykonaj symulacje zgodnie z punktem 1.11 - zrobione
- 5BIT DAC SCHEMATIC** - zrobione
- Stwórz symbol dla 5bit dac - zrobione
- Stwórz schemat symulacyjny - zrobione
- Przeprowadź symulacje działania układu - VTH jest na poziomie 1V dla $W/L = 2/5$ przez co nie jesteśmy w stanie uzyskać prądu bliskiego 10uA, nie bawić się w lvt-pfets, dla $W/L = 2/5$ $v_{gs} \approx 1.9V$ dla zasilania 1.8V X_X - modyfikacja W/L, obecnie 3.5/5, bez problemu można by było zostać przy $W = 2$ i zmniejszać L do momentu uzyskania odpowiedniego prądu
- Zastanów się jak zrobić symulacje cornerowe dla każdego przypadku (chodzi o każdy corner w jednej symulacji - czy jest to wogóle możliwe, jeśli nie - po prostu zmieniaj corner co symulacje) - zostałem przy drugiej opcji, pierwsza do sprawdzenia
- Layout DAC-a - TBD
- Co do samej instrukcji - dodać więcej zdjęć (łatwiej się pracuje z referencyjnymi zdjęciami) lub wstawiać pomocnicze fragmenty kodu (większość ustawiania to po prostu ngspice code)